

APEX Scenario and Scenario-Registry Usage Patterns

Date: 2025-01-24

Module: apex-demo/scenario

Purpose: Comprehensive documentation of scenario and scenario-registry patterns demonstrated in APEX

Overview

This document describes the various scenario and scenario-registry usage patterns demonstrated in the APEX demo module. Scenarios provide powerful workflow orchestration capabilities with stage-based processing, classification-driven routing, and comprehensive failure handling.

 **KEY CONCEPT:** APEX provides two distinct processing approaches:

1. **Automatic Routing** (`processMapData`) - Requires classification rules to determine which scenario should process incoming data
2. **Direct Processing** (`processDataWithStages`) - Executes a specific scenario by ID, no classification rules needed

When You Need Classification Rules:

- **Multiple processing routes** for different data types (e.g., OTC Options vs Bonds vs Swaps)
- **Automatic scenario selection** based on data content
- **Registry-based routing** where APEX decides which scenario to use

When You DON'T Need Classification Rules:

- **Single processing route** - only one scenario handles all data
- **Direct scenario execution** - application code specifies which scenario to use
- **Simple workflows** where routing logic is handled outside APEX

Scenario and Scenario-Registry Usage Patterns

1. Scenario Registry Pattern

File: `ScenarioEndToEndIntegrationTest.java` (Registry YAML)

```
# Scenario Registry - Central configuration index
metadata:
  id: "otc-option-registry"
  name: "OTC Option Scenario Registry"
  type: "scenario-registry"
  created-by: "test@example.com"

scenarios:
  - scenario-id: "otc-option-us"
    config-file: "otc-option-us-scenario.yaml"
    business-domain: "Derivatives Trading"
```

Registry Pattern Features:

- **Central Index:** Single file that references all available scenarios
- **Flexible Processing:** Supports both automatic routing and direct scenario execution
- **Business Domain:** Organizational grouping of scenarios for enterprise management
- **Scenario Discovery:** Automatic loading and registration of all scenarios in the registry

Two Processing Approaches:

1. **Automatic Routing** (`processMapData`):
 - Requires classification rules in scenario files
 - APEX automatically selects the appropriate scenario based on data content
 - Used when you have multiple scenarios and need intelligent routing
2. **Direct Processing** (`processDataWithStages`):
 - No classification rules required
 - Application specifies the scenario ID directly
 - Used when you know which scenario to execute or have only one processing route

Note: The `routing` section is optional. When omitted, APEX uses **classification-based routing** by default for automatic scenario selection.

Test Class: `ScenarioEndToEndIntegrationTest.java`

2. Stage-Based Scenario Processing Pattern (Direct Processing)

File: `BasicStageConfigurationTest-scenario.yaml`

This pattern demonstrates **direct scenario processing** where scenarios are executed by ID without requiring classification rules.

```
scenario:
  scenario-id: "basic-trade-processing"
  name: "Basic Trade Processing"

  # No classification rule needed for direct processing
  # Stage-based processing configuration
  processing-stages:
    # Stage 1: Validation (must pass)
    - stage-name: "validation"
      config-file: "BasicStageConfigurationTest-validation-rules.yaml"
      execution-order: 1
      failure-policy: "terminate"
      required: true
      depends-on: []
      stage-metadata:
        description: "Basic trade data validation"

    # Stage 2: Enrichment (optional)
    - stage-name: "enrichment"
      config-file: "BasicStageConfigurationTest-enrichment-rules.yaml"
      execution-order: 2
      failure-policy: "continue-with-warnings"
      required: false
      depends-on: ["validation"]
      stage-metadata:
        description: "Trade data enrichment"
```

Stage-Based Features:

- **Sequential Processing:** Stages execute in defined order
- **Dependency Management:** Stages can depend on other stages (depends-on)
- **Failure Policies:** Different error handling strategies per stage
- **SLA Tracking:** Performance monitoring with stage metadata
- **Required/Optional:** Control whether stages are mandatory
- **Direct Execution:** No classification rules needed - scenario executed by ID

Usage Example:

```
// Direct processing - no classification rules required
Map<String, Object> tradeData = new HashMap<>();
tradeData.put("tradeId", "TRADE-001");
tradeData.put("amount", 1000000);

// Execute specific scenario directly by ID
ScenarioExecutionResult result = scenarioService.processDataWithStages(tradeData, "basic-trade-processing");
assertEquals("basic-trade-processing", result.getScenarioId());
```

Test Class: BasicStageConfigurationTest.java

3. Classification-Based Routing Pattern (Automatic Processing)

File: ScenarioEndToEndIntegrationTest.java (Scenario YAML)

This pattern demonstrates **automatic scenario routing** where APEX selects the appropriate scenario based on data content.

```
scenario:
  scenario-id: "otc-option-us"
  name: "OTC Option US Processing"

# Classification rule - REQUIRED for automatic routing
classification-rule:
  condition: "#data['tradeType'] == 'OTCOption' && #data['region'] == 'US'"
  description: "US OTC option trades"

processing-stages:
  - stage-name: "validation"
    config-file: "otc-validation-rules.yaml"
    execution-order: 1
    failure-policy: "terminate"
    required: true
    stage-metadata:
      description: "Validate trade data"

  - stage-name: "enrichment"
    config-file: "otc-enrichment-rules.yaml"
    execution-order: 2
    failure-policy: "continue-with-warnings"
    required: false
    depends-on: ["validation"]
    stage-metadata:
      description: "Enrich trade data"
```

When Classification Rules Are Required:

- **Multiple Processing Routes:** When you have different scenarios for different data types (OTC Options, Bonds, Swaps, etc.)
- **Automatic Data Routing:** When APEX needs to determine which scenario should process incoming Map data
- **Business Logic Encoding:** Rules capture the business conditions that define when each scenario applies
- **Registry-Based Processing:** When using `processMapData()` for automatic scenario selection

Classification-Based Routing Features:

- **SpEL Expressions:** Powerful business rules using Spring Expression Language
- **Automatic Data Routing:** Incoming data automatically routed to matching scenarios
- **Content-Based Selection:** Scenarios selected based on actual data content and values
- **Multi-Condition Logic:** Complex AND/OR conditions for precise business rule matching
- **Field Access:** Direct access to data fields using `#data['fieldName']` syntax
- **Numeric Comparisons:** Support for `>`, `<`, `>=`, `<=`, `==` operations
- **String Operations:** Pattern matching, `contains`, `startsWith`, `endsWith` operations

Advanced Classification Examples:

```
# Simple field matching
condition: "#data['tradeType'] == 'OTCOption'"

# Multiple field AND conditions
condition: "#data['tradeType'] == 'OTCOption' && #data['region'] == 'US'"

# Numeric threshold conditions
condition: "#data['tradeType'] == 'OTCOption' && #data['notional'] > 100000000"

# String pattern matching
condition: "#data['instrumentId'].startsWith('USD') && #data['maturity'] > '2025-01-01'"

# Complex OR conditions
condition: "#data['assetClass'] == 'Equity' || #data['assetClass'] == 'Bond'"
```

Usage Example:

```
// Create test data matching classification rule
Map<String, Object> tradeData = new HashMap<>();
tradeData.put("tradeType", "OTCOption"); // Matches condition
tradeData.put("region", "US");           // Matches condition
tradeData.put("notional", 75000000);

// Automatic processing - APEX selects scenario based on classification rules
ScenarioExecutionResult result = scenarioService.processMapData(tradeData);
assertEquals("otc-option-us", result.getScenarioId());
```

When to Use This Pattern:

- **Multiple scenarios** with different processing logic for different data types
- **Dynamic routing** where the scenario depends on data content
- **Enterprise applications** with complex business domain routing requirements

 **KEY POINT:** Classification rules are required ONLY for automatic routing via `processMapData()`. For direct processing via `processDataWithStages(scenarioId)`, classification rules are optional.

Test Class: ScenarioEndToEndIntegrationTest.java

3.1. Multi-Scenario Classification Routing

File: ScenarioEndToEndIntegrationComplexTest.java

This pattern demonstrates how multiple scenarios with different classification rules work together in a single registry:

```
# Registry with multiple scenarios
scenarios:
  - scenario-id: "otc-option-us"
    config-file: "otc-option-us-scenario.yaml"
    business-domain: "Derivatives Trading"

  - scenario-id: "bond-us"
    config-file: "bond-us-scenario.yaml"
    business-domain: "Fixed Income"

# Scenario 1: OTC Options (MUST have classification rule)
scenario:
  scenario-id: "otc-option-us"
  classification-rule:
    condition: "#data['tradeType'] == 'OTCOption' && #data['assetClass'] == 'Derivative'"
    description: "OTC derivative option trades"

# Scenario 2: Bonds (MUST have classification rule)
scenario:
  scenario-id: "bond-us"
  classification-rule:
    condition: "#data['tradeType'] == 'Bond' && #data['assetClass'] == 'FixedIncome'"
    description: "Fixed income bond trades"
```

Multi-Scenario Features:

- **Scenario Precedence:** First matching scenario is selected (order matters)
- **Distinct Business Domains:** Different scenarios for different asset classes
- **Parallel Processing:** Each scenario has its own processing stages and rules
- **Automatic Selection:** APEX evaluates all classification rules and routes to the first match
- **Classification Required:** Every scenario MUST have a classification rule to participate in automatic routing

Test Class: ScenarioEndToEndIntegrationComplexTest.java

3.2. Routing Configuration (Advanced)

For advanced use cases, you can explicitly configure routing behavior in the registry:

```
# Optional explicit routing configuration
routing:
  strategy: "classification-based" # Default behavior
  default-scenario: "fallback-scenario" # Optional fallback

# Optional: Custom routing rules (rarely used)
rules:
  - condition: "#data['priority'] == 'HIGH'"
    target-scenario: "high-priority-scenario"
  - condition: "#data['region'] == 'APAC'"
```

```
target-scenario: "apac-scenario"
```

Routing Configuration Options:

- **strategy:** "classification-based" (default) or "type-based" (legacy)
- **default-scenario:** Fallback scenario when no classification rules match
- **rules:** Custom routing rules (rarely needed - use classification rules instead)

💡 **Best Practice:** In most cases, omit the `routing` section entirely and rely on embedded `classification-rule` conditions in scenario files. This provides cleaner, more maintainable configuration.

⚠️ **IMPORTANT:** When using automatic routing (`processMapData`), every scenario MUST have a classification rule. For direct processing (`processDataWithStages`), classification rules are optional since you specify the scenario ID explicitly.

4. Enhanced Data Classification Pattern

File: `InputDataClassificationPhase1Test.java`

```
// Enhanced scenario service with classification capabilities
EnhancedDataTypeScenarioService scenarioService = new EnhancedDataTypeScenarioService();
ApexEngine apexEngine = new ApexEngine(scenarioService);

// Content-based classification with caching
ApexProcessingContext context = ApexProcessingContext.builder()
    .source("test")
    .fileName("trade_message.json")
    .fileSize((long) jsonData.length())
    .addMetadata("region", "US")
    .build();

ApexProcessingResult result = apexEngine.classifyAndProcessData(jsonData, context);
ClassificationResult classification = result.getClassification();

// Enhanced confidence scoring
assertTrue(classification.getConfidence() > 0.6, "Should have good confidence");
assertEquals("json", classification.getFileFormat(), "Should detect JSON format");
```

Enhanced Classification Features:

- **Content-Based Detection:** Analyzes data content beyond file extensions
- **Confidence Scoring:** Multi-factor confidence calculation
- **Caching:** Performance optimization through classification caching
- **Format Detection:** Automatic detection of JSON, XML, CSV formats
- **Context Awareness:** Processing context with metadata

Test Class: `InputDataClassificationPhase1Test.java`

5. Validation Failure Handling Pattern

File: `ValidationFailureScenarioTest.java`

```

// Create trade data that violates business rules
Map<String, Object> tradeData = createTradeDataWithViolations();
tradeData.put("tradeId", "TRADE-001");           // Non-numeric (violates rule)
tradeData.put("instrumentType", "EQUITY");        // Not derivative (violates rule)
tradeData.put("quantity", 1000);                  // Too small (violates rule)
tradeData.put("currency", "USD");                  // Not exotic (violates rule)

// Execute scenario - validation rules trigger but processing continues
ScenarioExecutionResult result = scenarioService.processDataWithStages(
    tradeData, "validation-failure-scenario");

// Validation rules trigger (return true) when they detect violations
// Processing continues based on failure policy
assertTrue(validationResult.isSuccessful(),
    "Validation stage completes successfully even when rules trigger violations");

```

Validation Failure Features:

- **Informational Validation:** Rules detect violations without blocking
- **Comprehensive Reporting:** Detailed execution results with violation tracking
- **Configurable Policies:** Different responses to validation failures
- **Audit Trail:** Complete record of validation outcomes
- **Business Rule Detection:** Rules designed to trigger on violations

Test Class: ValidationFailureScenarioTest.java

6. End-to-End Integration Pattern

File: ScenarioEndToEndIntegrationTest.java

```

// Complete flow: Registry → Scenario → Validation → Enrichment

// 1. Create and load scenario registry
String registryYaml = createRegistryYaml();
Path registryFile = tempDir.resolve("registry.yaml");
Files.writeString(registryFile, registryYaml);
scenarioService.loadScenarios(registryFile.toString());

// 2. Create test data matching classification rule
Map<String, Object> tradeData = new HashMap<>();
tradeData.put("tradeType", "OTCOption");
tradeData.put("region", "US");
tradeData.put("notional", 75000000);

// 3. Process with automatic scenario selection
ScenarioExecutionResult result = scenarioService.processMapData(tradeData);

// 4. Validate complete execution
assertEquals("otc-option-us", result.getScenarioId());
assertTrue(result.isSuccessful());
assertFalse(result.isTerminated());

// 5. Verify stage execution
List<String> executedStages = result.getStageResults().stream()
    .map(StageExecutionResult::getStageName)
    .toList();
assertTrue(executedStages.contains("validation"));
assertTrue(executedStages.contains("enrichment"));

```

End-to-End Features:

- **Complete Workflow:** Registry loading through final processing
- **Automatic Selection:** Classification rules route data to scenarios
- **Multi-Stage Execution:** Sequential processing with dependency management
- **Comprehensive Results:** Detailed execution tracking and reporting
- **Dynamic Configuration:** Runtime creation of YAML configurations

Test Class: `ScenarioEndToEndIntegrationTest.java`



Pattern Summary Table

Pattern	Purpose	Key Features	Usage Frequency	Files Demonstrated
Scenario Registry	Central configuration index	Registry files, scenario discovery, business domains	Always	ScenarioEndToEndIntegrationTest.java
Stage-Based Processing	Sequential workflow execution	Execution order, dependencies, failure policies	Always	BasicStageConfigurationTest-scenario.yaml
Classification-Based Routing	Automatic scenario selection	SpEL expressions, content-based routing	99% of use cases	ScenarioEndToEndIntegrationTest.java
Multi-Scenario Classification	Multiple scenario routing	Scenario precedence, distinct business domains	Common	ScenarioEndToEndIntegrationComplexTest.java
Enhanced Classification	Intelligent data detection	Content analysis, confidence scoring, caching	Advanced	InputDataClassificationPhase1Test.java
Validation Failure Handling	Business rule violation management	Informational validation, comprehensive reporting	Common	ValidationFailureScenarioTest.java
End-to-End Integration	Complete workflow demonstration	Full pipeline from registry to results	Testing	ScenarioEndToEndIntegrationTest.java

Key APEX Scenario Capabilities

Failure Policies

- `terminate` : Stop processing immediately on critical failures
- `continue-with-warnings` : Log warnings but continue processing
- `flag-for-review` : Mark for manual review while continuing

Classification Strategies

- **Classification-Based** (Primary): Use SpEL expressions for intelligent content-based routing - **99% of use cases**
- **Type-Based** (Legacy): Route based on Java class names - rarely used, mainly for backward compatibility

Note: Type-based routing is primarily used for legacy Java object routing and is not commonly used in modern APEX applications that process Map-based data.

Stage Dependencies

- **Sequential Execution:** Stages run in defined order (`execution-order`)
- **Dependency Management:** Stages can depend on successful completion of other stages (`depends-on`)
- **Conditional Execution:** Stages only run if dependencies are met

Performance Features

- **SLA Tracking:** Stage-level performance monitoring (`sla-ms`)
- **Execution Metrics:** Comprehensive timing and success tracking
- **Caching:** Classification result caching for performance optimization

Best Practices

1. Modular Design

Separate registry, scenario, and rule files for better maintainability and team collaboration.

2. Choose the Right Processing Approach

- **Use automatic routing** (`processMapData`) when you have multiple scenarios and need intelligent data routing based on content
- **Use direct processing** (`processDataWithStages`) when you have a single processing route or when application logic determines the scenario
- **Classification rules are required only for automatic routing** - they're optional for direct processing

3. Failure Resilience

Configure appropriate failure policies per stage:

- Use `terminate` for critical validations
- Use `continue-with-warnings` for optional enrichments

- Use `flag-for-review` for complex business decisions

4. Performance Optimization

- Leverage classification caching for repeated data processing
- Set appropriate SLA targets for stage monitoring
- Use content-based detection for accurate classification

5. Comprehensive Testing

- Test both positive and negative scenarios
- Validate failure policy enforcement
- Test end-to-end workflows with real data

6. Audit Trail

Maintain detailed execution results for compliance and debugging.



Running the Examples

```
▶# Run basic stage configuration tests
mvn test -Dtest=BasicStageConfigurationTest -pl apex-demo
▶
# Run classification tests
mvn test -Dtest=InputDataClassificationPhase1Test -pl apex-demo
▶
# Run validation failure tests
mvn test -Dtest=ValidationFailureScenarioTest -pl apex-demo
▶
# Run end-to-end integration tests
mvn test -Dtest=ScenarioEndToEndIntegrationTest -pl apex-demo
▶
# Run all scenario tests
mvn test -Dtest=*Scenario*Test -pl apex-demo
▶
```



Conclusion

The scenario patterns in APEX provide a comprehensive framework for orchestrating complex data processing workflows. From simple stage-based processing to sophisticated classification-driven routing, these patterns enable scalable, maintainable scenario management systems suitable for enterprise applications.

The demonstrated patterns show APEX's flexibility in handling everything from basic sequential processing to complex multi-stage workflows with intelligent data routing, making it suitable for both simple applications and large-scale enterprise scenario management systems.